# ОТЧЁТ

Элтекс. Модуль 1. Практика 3

Процессы, сигналы, задания, cron и systemd

Выполнил: Береснев Ю.Н. (логин на сервере практики 1: eltex-pg1-v7)

---

## Раздел 1. Процессы, сигналы и задания

### 1.1. Вход под user1 и многопоточные процессы

**Команда:**

```
su - user1
```

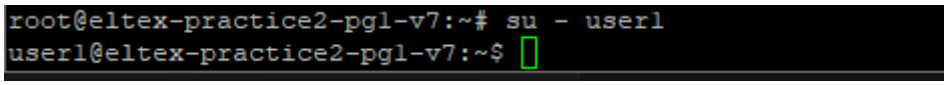



Рисунок 1 – Вход под пользователем user1

**Команда:**

```
ps -eLo pid,nlwp | awk '$2 > 1 {count++} END {print "Процессов с >1 потоком: "
count}'
```

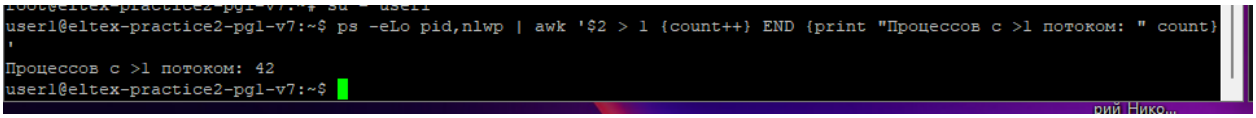



Рисунок 2 – Подсчёт процессов, имеющих несколько потоков выполнения

***Комментарий:*** *ps выводит PID и число потоков (nlwp), awk считает записи, в которых число потоков больше 1. Результат: 42.*

## 1.2. Утилита top: сортировка и настройка полей

**Команда:**

```
top
```

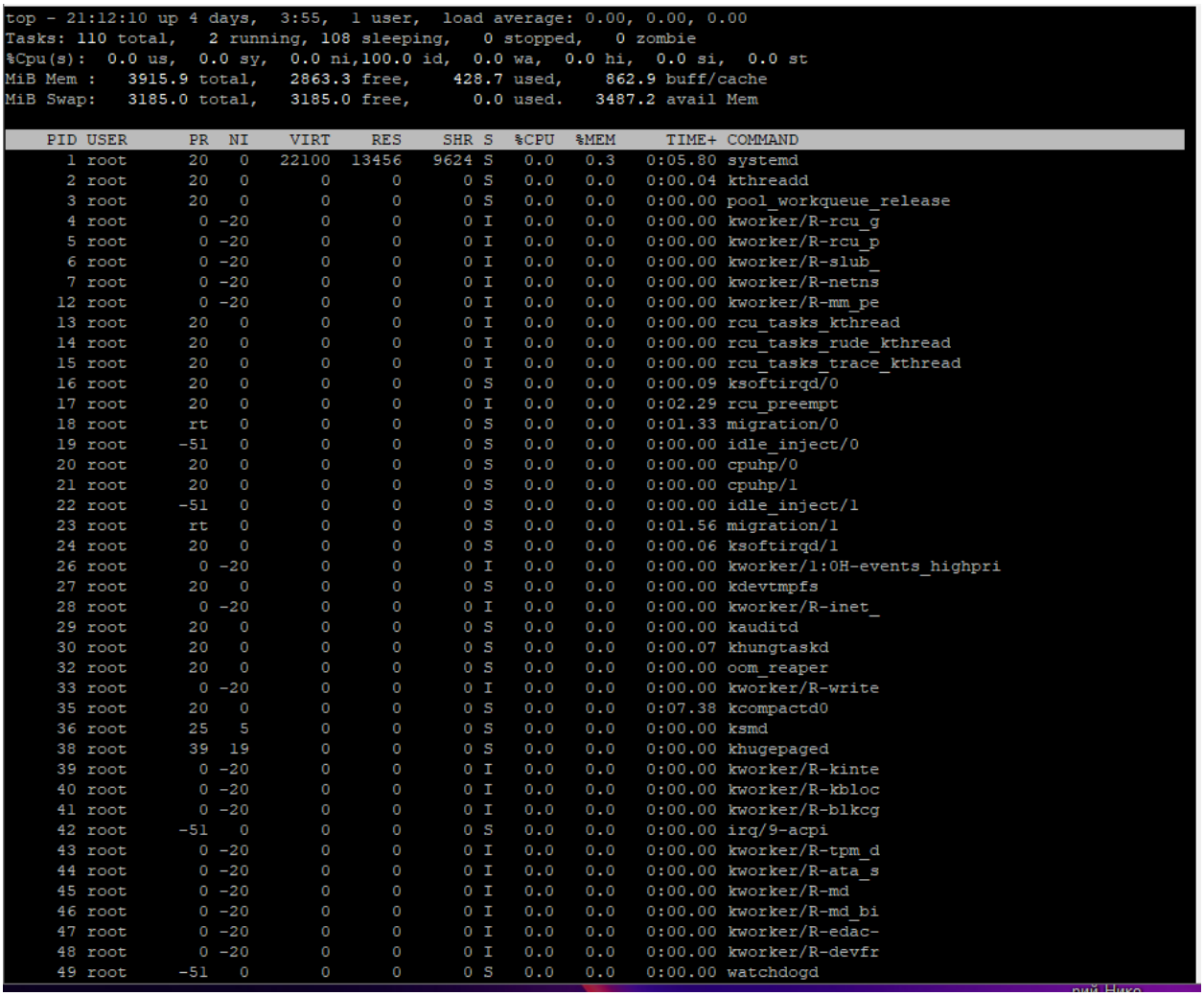
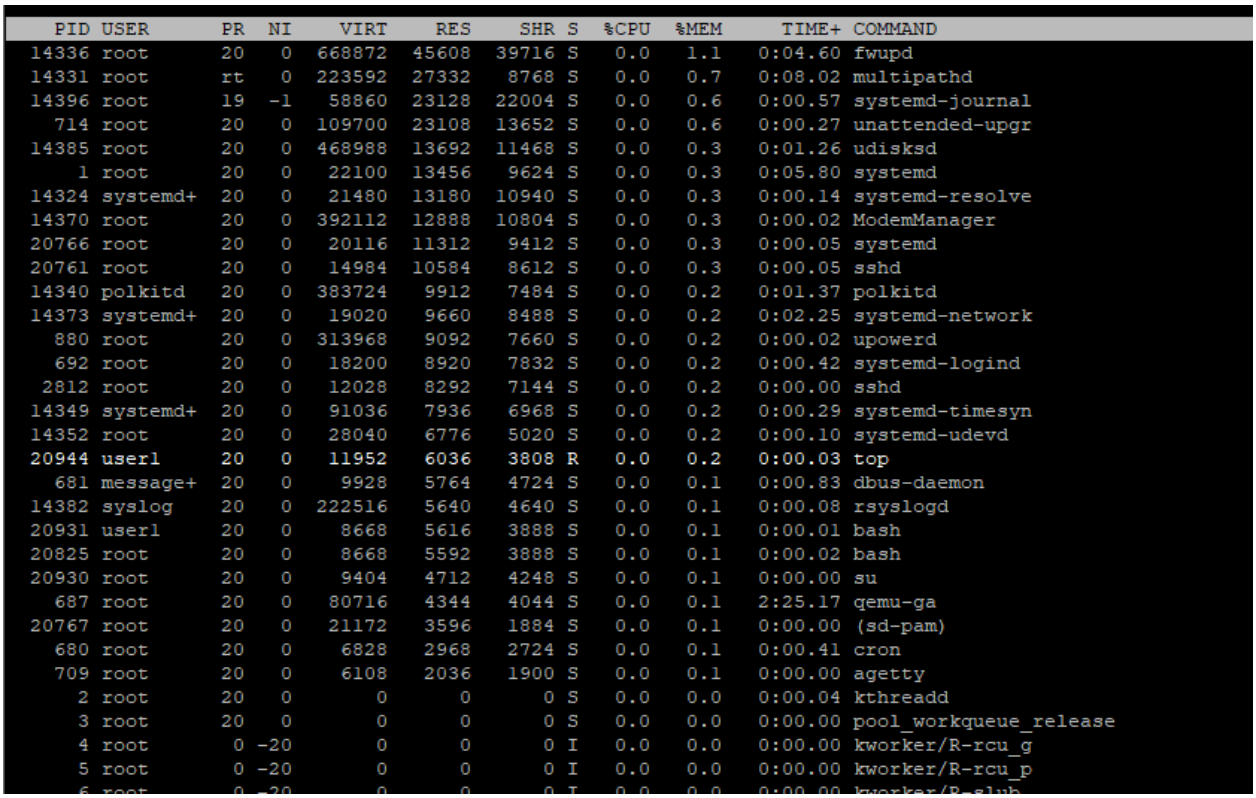



Рисунок 3 – Запуск top под пользователем user1

***Комментарий:*** *Стандартный вывод: поля PID, USER, PR, NI, VIRT, RES, SHR, S, %CPU, %MEM, TIME+, COMMAND.*

**Клавиши top:**

```
Shift+M
```



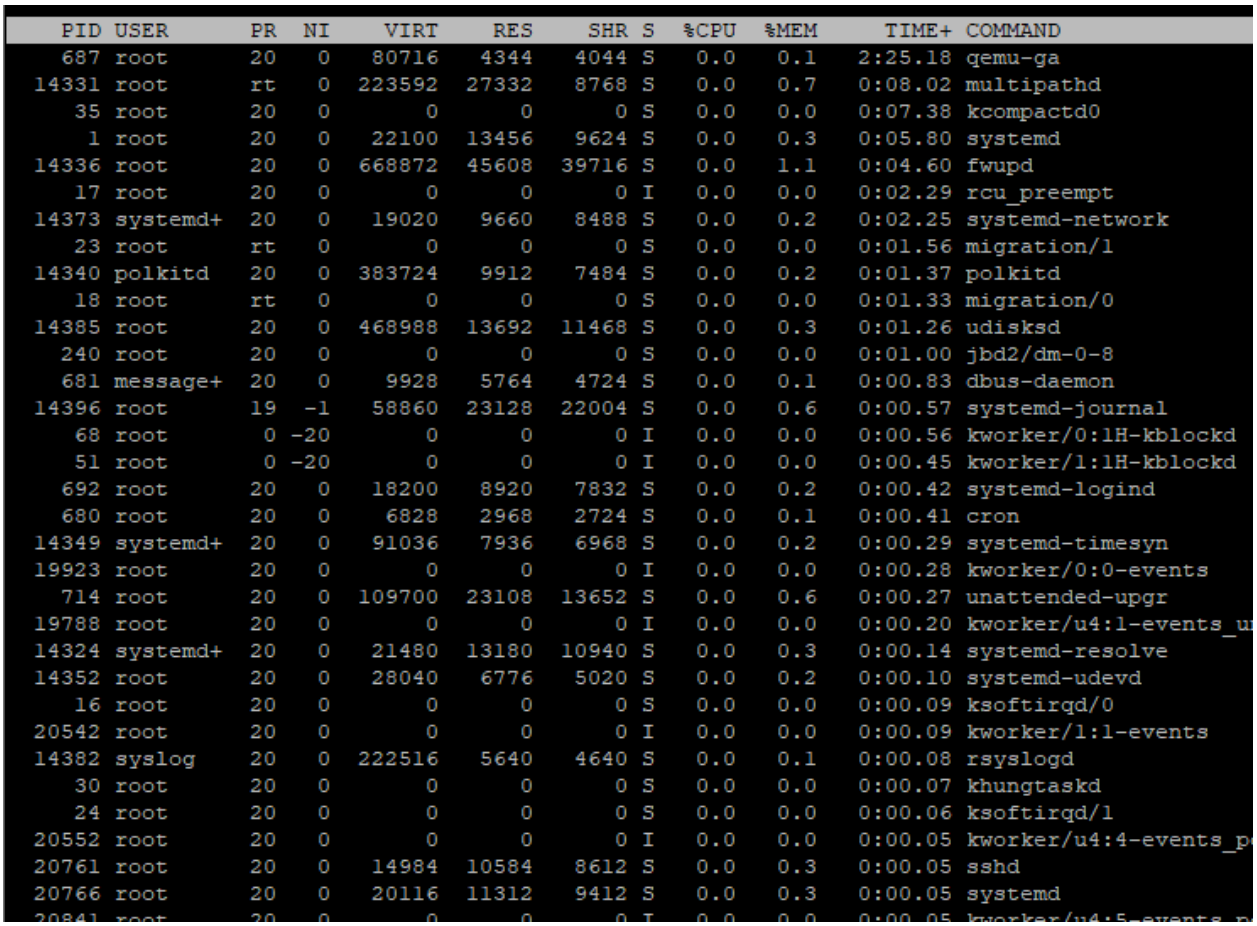
```
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  14336 root      20   0  668872  45608  39716 S   0.0   1.1   0:04.60 fwupd
  14331 root      rt   0  223592  27332   8768 S   0.0   0.7   0:08.02 multipathd
  14396 root      19  -1   58860  23128  22004 S   0.0   0.6   0:00.57 systemd-journal
    714 root      20   0  109700  23108  13652 S   0.0   0.6   0:00.27 unattended-upgr
  14385 root      20   0  468988  13692  11468 S   0.0   0.3   0:01.26 udisksd
      1 root      20   0   22100  13456   9624 S   0.0   0.3   0:05.80 systemd
  14324 systemd+  20   0   21480  13180  10940 S   0.0   0.3   0:00.14 systemd-resolve
  14370 root      20   0  392112  12888  10804 S   0.0   0.3   0:00.02 ModemManager
  20766 root      20   0   20116  11312   9412 S   0.0   0.3   0:00.05 systemd
  20761 root      20   0   14984  10584   8612 S   0.0   0.3   0:00.05 sshd
  14340 polkitd   20   0  383724   9912   7484 S   0.0   0.2   0:01.37 polkitd
  14373 systemd+  20   0   19020   9660   8488 S   0.0   0.2   0:02.25 systemd-network
    880 root      20   0  313968   9092   7660 S   0.0   0.2   0:00.02 upowerd
    692 root      20   0   18200   8920   7832 S   0.0   0.2   0:00.42 systemd-logind
   2812 root      20   0   12028   8292   7144 S   0.0   0.2   0:00.00 sshd
  14349 systemd+  20   0   91036   7936   6968 S   0.0   0.2   0:00.29 systemd-timesyn
  14352 root      20   0   28040   6776   5020 S   0.0   0.2   0:00.10 systemd-udevd
  20944 userl     20   0   11952   6036   3808 R   0.0   0.2   0:00.03 top
    681 message+  20   0    9928   5764   4724 S   0.0   0.1   0:00.83 dbus-daemon
  14382 syslog    20   0  222516   5640   4640 S   0.0   0.1   0:00.08 rsyslogd
  20931 userl     20   0    8668   5616   3888 S   0.0   0.1   0:00.01 bash
  20825 root      20   0    8668   5592   3888 S   0.0   0.1   0:00.02 bash
  20930 root      20   0    9404   4712   4248 S   0.0   0.1   0:00.00 su
    687 root      20   0   80716   4344   4044 S   0.0   0.1   2:25.17 qemu-ga
  20767 root      20   0   21172   3596   1884 S   0.0   0.1   0:00.00 (sd-pam)
    680 root      20   0    6828   2968   2724 S   0.0   0.1   0:00.41 cron
    709 root      20   0    6108   2036   1900 S   0.0   0.1   0:00.00 agetty
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.04 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub
```

Рисунок 4 – Сортировка по использованию памяти (поле %MEM)

***Комментарий:*** *Порядок процессов соответствует убыванию RES (резидентной памяти). RES — объём физической оперативной памяти, который процесс занимает в данный момент.*

**Клавиши top:**

```
Shift+T
```

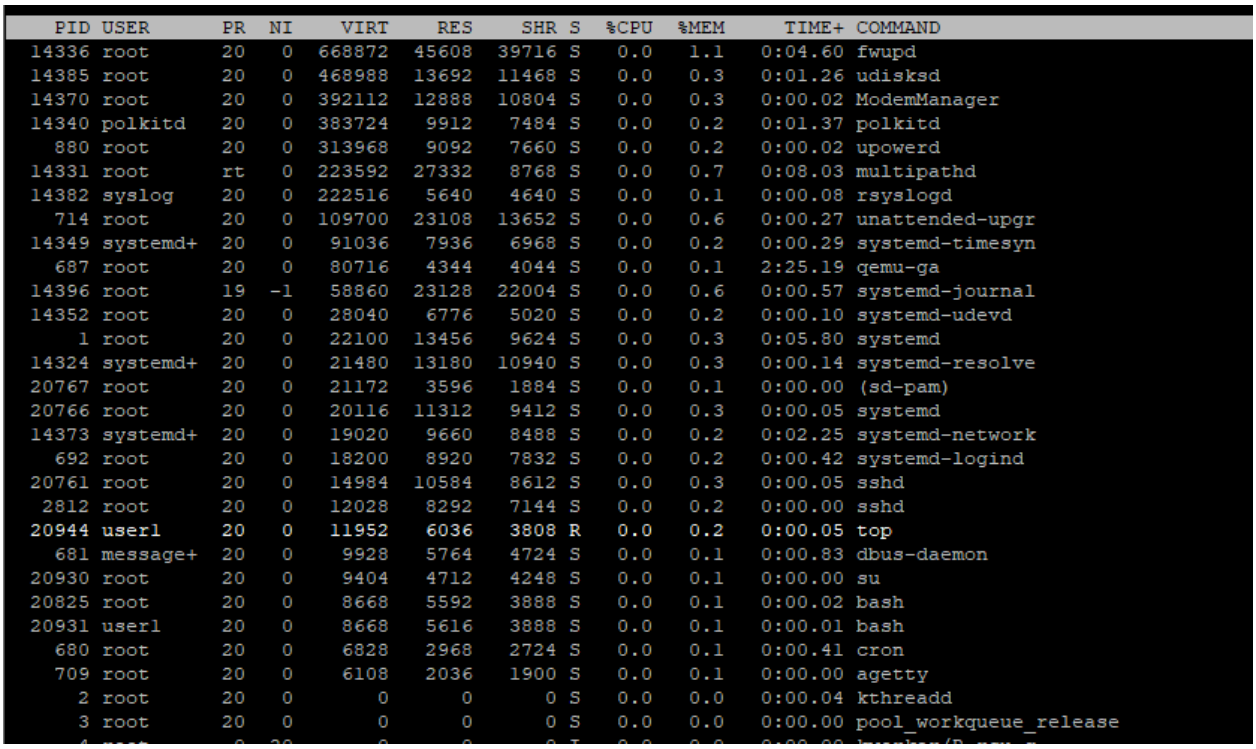


```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  687 root      20   0   80716   4344   4044 S   0.0   0.1   2:25.18 qemu-ga
14331 root      rt   0  223592  27332   8768 S   0.0   0.7   0:08.02 multipathd
   35 root      20   0       0      0      0 S   0.0   0.0   0:07.38 kcompactd0
    1 root      20   0   22100  13456   9624 S   0.0   0.3   0:05.80 systemd
14336 root      20   0  668872  45608  39716 S   0.0   1.1   0:04.60 fwupd
   17 root      20   0       0      0      0 I   0.0   0.0   0:02.29 rcu_preempt
14373 systemd+  20   0   19020   9660   8488 S   0.0   0.2   0:02.25 systemd-network
   23 root      rt   0       0      0      0 S   0.0   0.0   0:01.56 migration/1
14340 polkitd   20   0  383724   9912   7484 S   0.0   0.2   0:01.37 polkitd
   18 root      rt   0       0      0      0 S   0.0   0.0   0:01.33 migration/0
14385 root      20   0  468988  13692  11468 S   0.0   0.3   0:01.26 udisksd
  240 root      20   0       0      0      0 S   0.0   0.0   0:01.00 jbd2/dm-0-8
  681 message+  20   0    9928   5764   4724 S   0.0   0.1   0:00.83 dbus-daemon
14396 root      19  -1   58860  23128  22004 S   0.0   0.6   0:00.57 systemd-journal
   68 root       0 -20       0      0      0 I   0.0   0.0   0:00.56 kworker/0:1H-kblockd
   51 root       0 -20       0      0      0 I   0.0   0.0   0:00.45 kworker/1:1H-kblockd
  692 root      20   0   18200   8920   7832 S   0.0   0.2   0:00.42 systemd-logind
  680 root      20   0    6828   2968   2724 S   0.0   0.1   0:00.41 cron
14349 systemd+  20   0   91036   7936   6968 S   0.0   0.2   0:00.29 systemd-timesyn
19923 root      20   0       0      0      0 I   0.0   0.0   0:00.28 kworker/0:0-events
  714 root      20   0  109700  23108  13652 S   0.0   0.6   0:00.27 unattended-upgr
19788 root      20   0       0      0      0 I   0.0   0.0   0:00.20 kworker/u4:1-events_u
14324 systemd+  20   0   21480  13180  10940 S   0.0   0.3   0:00.14 systemd-resolve
14352 root      20   0   28040   6776   5020 S   0.0   0.2   0:00.10 systemd-udevd
   16 root      20   0       0      0      0 S   0.0   0.0   0:00.09 ksoftirqd/0
20542 root      20   0       0      0      0 I   0.0   0.0   0:00.09 kworker/1:1-events
14382 syslog    20   0  222516   5640   4640 S   0.0   0.1   0:00.08 rsyslogd
   30 root      20   0       0      0      0 S   0.0   0.0   0:00.07 khungtaskd
   24 root      20   0       0      0      0 S   0.0   0.0   0:00.06 ksoftirqd/1
20552 root      20   0       0      0      0 I   0.0   0.0   0:00.05 kworker/u4:4-events_p
20761 root      20   0   14984  10584   8612 S   0.0   0.3   0:00.05 sshd
20766 root      20   0   20116  11312   9412 S   0.0   0.3   0:00.05 systemd
```

Рисунок 5 – Сортировка по накопленному процессорному времени (поле TIME+)

**Действие в top:**

```
сортировка по полю VIRT
```



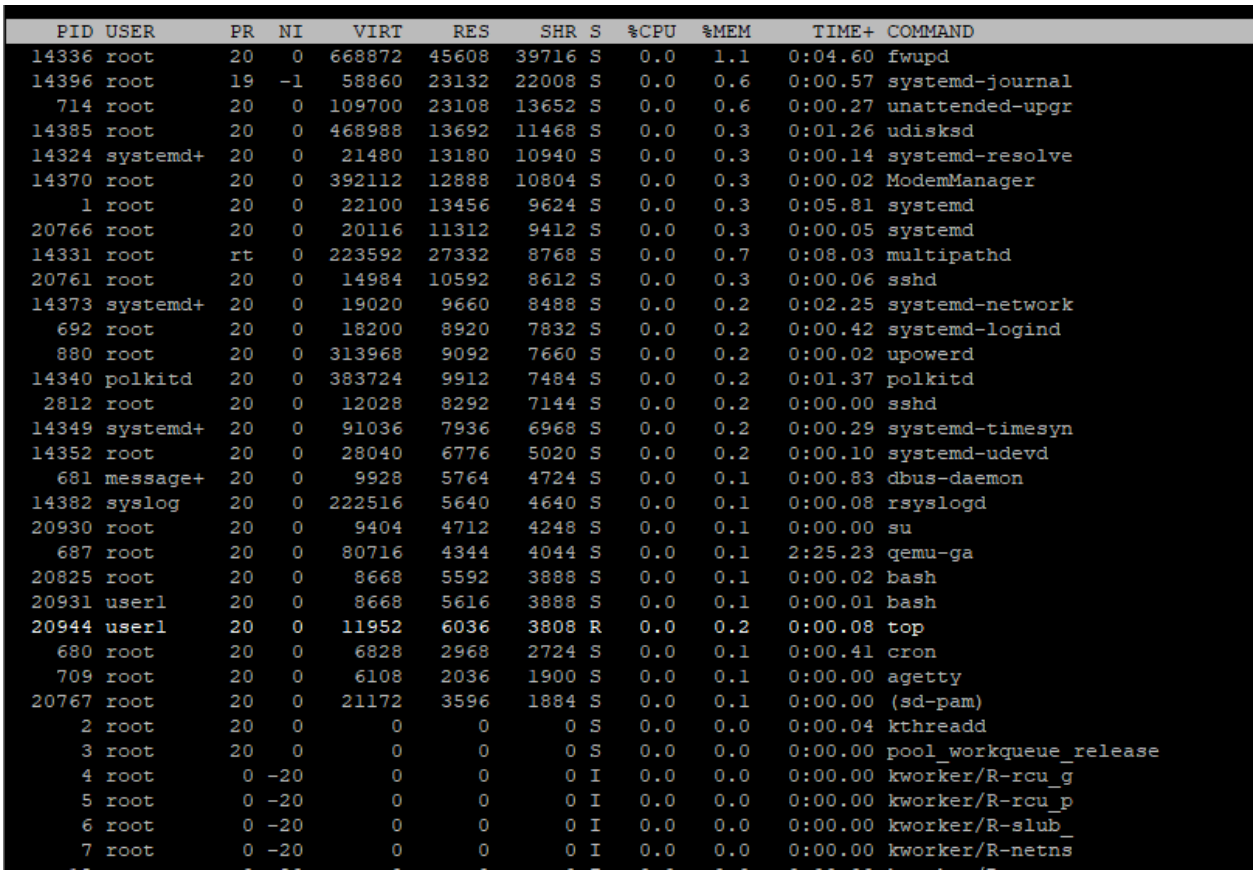
| PID | USER | PR | NI | VIRT | RES | SHR | S | %CPU | %MEM | TIME+ | COMMAND |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 14336 | root | 20 | 0 | 668872 | 45608 | 39716 | S | 0.0 | 1.1 | 0:04.60 | fwupd |
| 14385 | root | 20 | 0 | 468988 | 13692 | 11468 | S | 0.0 | 0.3 | 0:01.26 | udisksd |
| 14370 | root | 20 | 0 | 392112 | 12888 | 10804 | S | 0.0 | 0.3 | 0:00.02 | ModemManager |
| 14340 | polkitd | 20 | 0 | 383724 | 9912 | 7484 | S | 0.0 | 0.2 | 0:01.37 | polkitd |
| 880 | root | 20 | 0 | 313968 | 9092 | 7660 | S | 0.0 | 0.2 | 0:00.02 | upowerd |
| 14331 | root | rt | 0 | 223592 | 27332 | 8768 | S | 0.0 | 0.7 | 0:08.03 | multipathd |
| 14382 | syslog | 20 | 0 | 222516 | 5640 | 4640 | S | 0.0 | 0.1 | 0:00.08 | rsyslogd |
| 714 | root | 20 | 0 | 109700 | 23108 | 13652 | S | 0.0 | 0.6 | 0:00.27 | unattended-upgr |
| 14349 | systemd+ | 20 | 0 | 91036 | 7936 | 6968 | S | 0.0 | 0.2 | 0:00.29 | systemd-timesyn |
| 687 | root | 20 | 0 | 80716 | 4344 | 4044 | S | 0.0 | 0.1 | 2:25.19 | qemu-ga |
| 14396 | root | 19 | -1 | 58860 | 23128 | 22004 | S | 0.0 | 0.6 | 0:00.57 | systemd-journal |
| 14352 | root | 20 | 0 | 28040 | 6776 | 5020 | S | 0.0 | 0.2 | 0:00.10 | systemd-udevd |
| 1 | root | 20 | 0 | 22100 | 13456 | 9624 | S | 0.0 | 0.3 | 0:05.80 | systemd |
| 14324 | systemd+ | 20 | 0 | 21480 | 13180 | 10940 | S | 0.0 | 0.3 | 0:00.14 | systemd-resolve |
| 20767 | root | 20 | 0 | 21172 | 3596 | 1884 | S | 0.0 | 0.1 | 0:00.00 | (sd-pam) |
| 20766 | root | 20 | 0 | 20116 | 11312 | 9412 | S | 0.0 | 0.3 | 0:00.05 | systemd |
| 14373 | systemd+ | 20 | 0 | 19020 | 9660 | 8488 | S | 0.0 | 0.2 | 0:02.25 | systemd-network |
| 692 | root | 20 | 0 | 18200 | 8920 | 7832 | S | 0.0 | 0.2 | 0:00.42 | systemd-logind |
| 20761 | root | 20 | 0 | 14984 | 10584 | 8612 | S | 0.0 | 0.3 | 0:00.05 | sshd |
| 2812 | root | 20 | 0 | 12028 | 8292 | 7144 | S | 0.0 | 0.2 | 0:00.00 | sshd |
| **20944** | **user1** | **20** | **0** | **11952** | **6036** | **3808** | **R** | **0.0** | **0.2** | **0:00.05** | **top** |
| 681 | message+ | 20 | 0 | 9928 | 5764 | 4724 | S | 0.0 | 0.1 | 0:00.83 | dbus-daemon |
| 20930 | root | 20 | 0 | 9404 | 4712 | 4248 | S | 0.0 | 0.1 | 0:00.00 | su |
| 20825 | root | 20 | 0 | 8668 | 5592 | 3888 | S | 0.0 | 0.1 | 0:00.02 | bash |
| 20931 | user1 | 20 | 0 | 8668 | 5616 | 3888 | S | 0.0 | 0.1 | 0:00.01 | bash |
| 680 | root | 20 | 0 | 6828 | 2968 | 2724 | S | 0.0 | 0.1 | 0:00.41 | cron |
| 709 | root | 20 | 0 | 6108 | 2036 | 1900 | S | 0.0 | 0.1 | 0:00.00 | agetty |
| 2 | root | 20 | 0 | 0 | 0 | 0 | S | 0.0 | 0.0 | 0:00.04 | kthreadd |
| 3 | root | 20 | 0 | 0 | 0 | 0 | S | 0.0 | 0.0 | 0:00.00 | pool_workqueue_release |

Рисунок 6 – Сортировка процессов по убыванию VIRT

***Комментарий:*** *VIRT — общий объём виртуальной памяти, которую процесс может использовать: код, данные, библиотеки, отображённые файлы и память в swap. Он обычно намного больше физической памяти (RES).*

**Действие в top:**

```
сортировка по полю SHR
```



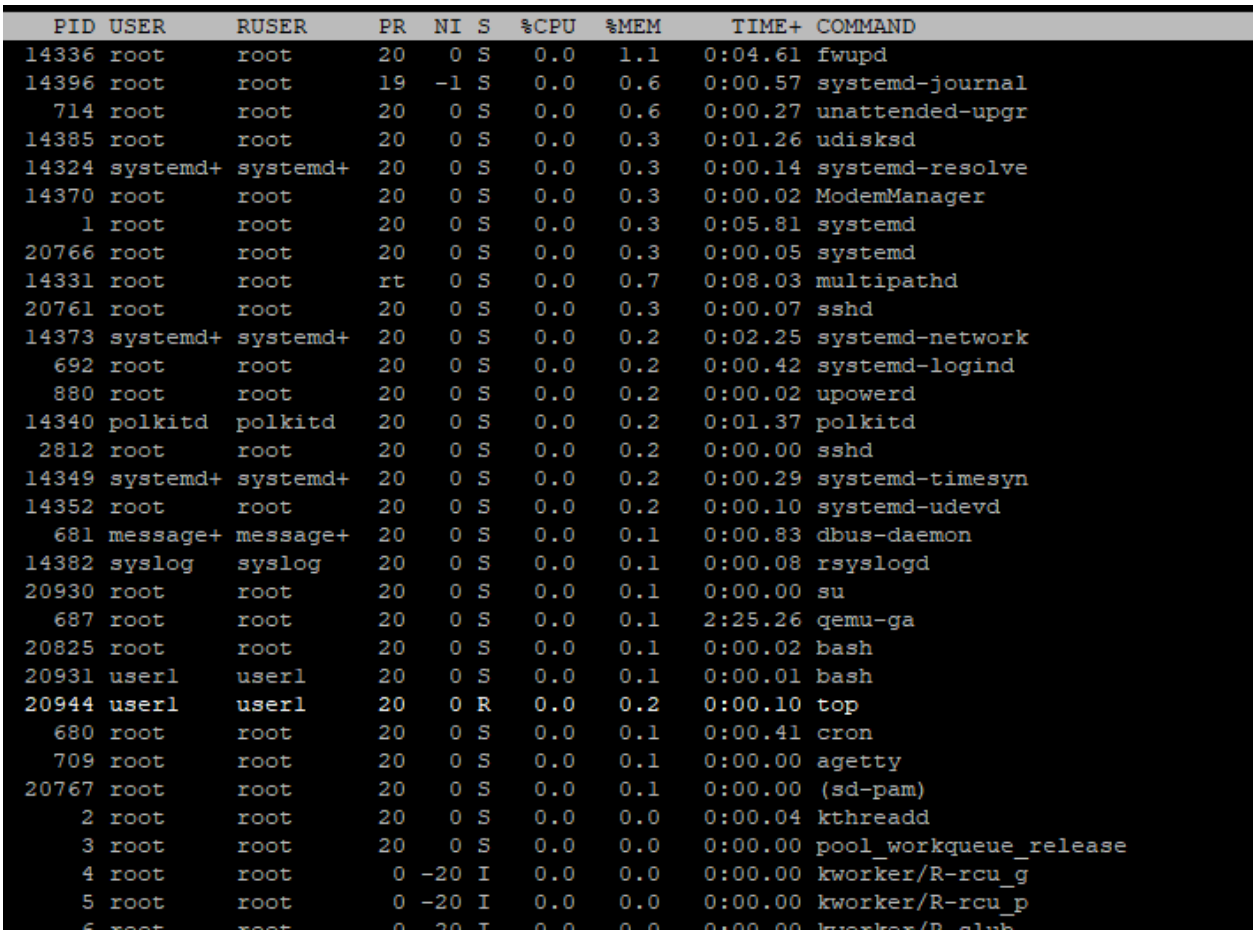
```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
14336 root      20   0  668872  45608  39716 S   0.0   1.1   0:04.60 fwupd
14396 root      19  -1   58860  23132  22008 S   0.0   0.6   0:00.57 systemd-journal
  714 root      20   0  109700  23108  13652 S   0.0   0.6   0:00.27 unattended-upgr
14385 root      20   0  468988  13692  11468 S   0.0   0.3   0:01.26 udisksd
14324 systemd+  20   0   21480  13180  10940 S   0.0   0.3   0:00.14 systemd-resolve
14370 root      20   0  392112  12888  10804 S   0.0   0.3   0:00.02 ModemManager
    1 root      20   0   22100  13456   9624 S   0.0   0.3   0:05.81 systemd
20766 root      20   0   20116  11312   9412 S   0.0   0.3   0:00.05 systemd
14331 root      rt   0  223592  27332   8768 S   0.0   0.7   0:08.03 multipathd
20761 root      20   0   14984  10592   8612 S   0.0   0.3   0:00.06 sshd
14373 systemd+  20   0   19020   9660   8488 S   0.0   0.2   0:02.25 systemd-network
  692 root      20   0   18200   8920   7832 S   0.0   0.2   0:00.42 systemd-logind
  880 root      20   0  313968   9092   7660 S   0.0   0.2   0:00.02 upowerd
14340 polkitd   20   0  383724   9912   7484 S   0.0   0.2   0:01.37 polkitd
 2812 root      20   0   12028   8292   7144 S   0.0   0.2   0:00.00 sshd
14349 systemd+  20   0   91036   7936   6968 S   0.0   0.2   0:00.29 systemd-timesyn
14352 root      20   0   28040   6776   5020 S   0.0   0.2   0:00.10 systemd-udevd
  681 message+  20   0    9928   5764   4724 S   0.0   0.1   0:00.83 dbus-daemon
14382 syslog    20   0  222516   5640   4640 S   0.0   0.1   0:00.08 rsyslogd
20930 root      20   0    9404   4712   4248 S   0.0   0.1   0:00.00 su
  687 root      20   0   80716   4344   4044 S   0.0   0.1   2:25.23 qemu-ga
20825 root      20   0    8668   5592   3888 S   0.0   0.1   0:00.02 bash
20931 user1     20   0    8668   5616   3888 S   0.0   0.1   0:00.01 bash
20944 user1     20   0   11952   6036   3808 R   0.0   0.2   0:00.08 top
  680 root      20   0    6828   2968   2724 S   0.0   0.1   0:00.41 cron
  709 root      20   0    6108   2036   1900 S   0.0   0.1   0:00.00 agetty
20767 root      20   0   21172   3596   1884 S   0.0   0.1   0:00.00 (sd-pam)
    2 root      20   0       0      0      0 S   0.0   0.0   0:00.04 kthreadd
    3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
    4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
    7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
```

Рисунок 7 – Сортировка процессов по убыванию SHR

***Комментарий:*** *SHR — объём разделяемой памяти процесса: общие библиотеки и области, которые могут одновременно использоваться несколькими процессами.*

**Действие в top:**

```
удаление полей VIRT, RES, SHR; добавление поля RUSER после USER (меню F)
```



| PID | USER | RUSER | PR | NI | S | %CPU | %MEM | TIME+ | COMMAND |
|---|---|---|---|---|---|---|---|---|---|
| 14336 | root | root | 20 | 0 | S | 0.0 | 1.1 | 0:04.61 | fwupd |
| 14396 | root | root | 19 | -1 | S | 0.0 | 0.6 | 0:00.57 | systemd-journal |
| 714 | root | root | 20 | 0 | S | 0.0 | 0.6 | 0:00.27 | unattended-upgr |
| 14385 | root | root | 20 | 0 | S | 0.0 | 0.3 | 0:01.26 | udisksd |
| 14324 | systemd+ | systemd+ | 20 | 0 | S | 0.0 | 0.3 | 0:00.14 | systemd-resolve |
| 14370 | root | root | 20 | 0 | S | 0.0 | 0.3 | 0:00.02 | ModemManager |
| 1 | root | root | 20 | 0 | S | 0.0 | 0.3 | 0:05.81 | systemd |
| 20766 | root | root | 20 | 0 | S | 0.0 | 0.3 | 0:00.05 | systemd |
| 14331 | root | root | rt | 0 | S | 0.0 | 0.7 | 0:08.03 | multipathd |
| 20761 | root | root | 20 | 0 | S | 0.0 | 0.3 | 0:00.07 | sshd |
| 14373 | systemd+ | systemd+ | 20 | 0 | S | 0.0 | 0.2 | 0:02.25 | systemd-network |
| 692 | root | root | 20 | 0 | S | 0.0 | 0.2 | 0:00.42 | systemd-logind |
| 880 | root | root | 20 | 0 | S | 0.0 | 0.2 | 0:00.02 | upowerd |
| 14340 | polkitd | polkitd | 20 | 0 | S | 0.0 | 0.2 | 0:01.37 | polkitd |
| 2812 | root | root | 20 | 0 | S | 0.0 | 0.2 | 0:00.00 | sshd |
| 14349 | systemd+ | systemd+ | 20 | 0 | S | 0.0 | 0.2 | 0:00.29 | systemd-timesyn |
| 14352 | root | root | 20 | 0 | S | 0.0 | 0.2 | 0:00.10 | systemd-udevd |
| 681 | message+ | message+ | 20 | 0 | S | 0.0 | 0.1 | 0:00.83 | dbus-daemon |
| 14382 | syslog | syslog | 20 | 0 | S | 0.0 | 0.1 | 0:00.08 | rsyslogd |
| 20930 | root | root | 20 | 0 | S | 0.0 | 0.1 | 0:00.00 | su |
| 687 | root | root | 20 | 0 | S | 0.0 | 0.1 | 2:25.26 | qemu-ga |
| 20825 | root | root | 20 | 0 | S | 0.0 | 0.1 | 0:00.02 | bash |
| 20931 | user1 | user1 | 20 | 0 | S | 0.0 | 0.1 | 0:00.01 | bash |
| 20944 | user1 | user1 | 20 | 0 | R | 0.0 | 0.2 | 0:00.10 | top |
| 680 | root | root | 20 | 0 | S | 0.0 | 0.1 | 0:00.41 | cron |
| 709 | root | root | 20 | 0 | S | 0.0 | 0.1 | 0:00.00 | agetty |
| 20767 | root | root | 20 | 0 | S | 0.0 | 0.1 | 0:00.00 | (sd-pam) |
| 2 | root | root | 20 | 0 | S | 0.0 | 0.0 | 0:00.04 | kthreadd |
| 3 | root | root | 20 | 0 | S | 0.0 | 0.0 | 0:00.00 | pool_workqueue_release |
| 4 | root | root | 0 | -20 | I | 0.0 | 0.0 | 0:00.00 | kworker/R-rcu_g |
| 5 | root | root | 0 | -20 | I | 0.0 | 0.0 | 0:00.00 | kworker/R-rcu_p |

Рисунок 8 – Настройка полей: удалены VIRT, RES, SHR, добавлено RUSER после USER

***Комментарий:*** *USER — эффективный пользователь, от имени которого процесс работает и по правам которого проверяется доступ. RUSER — реальный пользователь, запустивший процесс. Они различаются у процессов с SUID-битом (например, passwd).*

## 1.3. Процесс passwd и сигналы

**Команды:**

```
su - user1
passwd
```

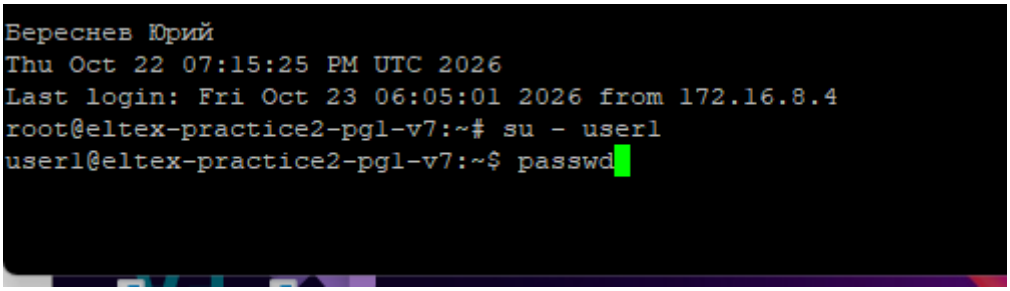



Рисунок 9 – Запуск passwd в отдельной сессии screen

***Комментарий:*** *Во втором терминале выполнено подключение по ssh к серверу практики 2 и запущен passwd под пользователем user1.*

**Команда:**

```
passwd
```

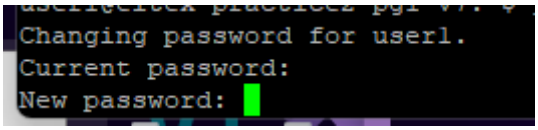



Рисунок 10 – Запрос пароля: введён текущий пароль, новый не вводится

***Комментарий:*** *Процесс passwd остаётся в состоянии ожидания ввода нового пароля.*

**Команды:**

```
Ctrl+a d
screen -ls
```

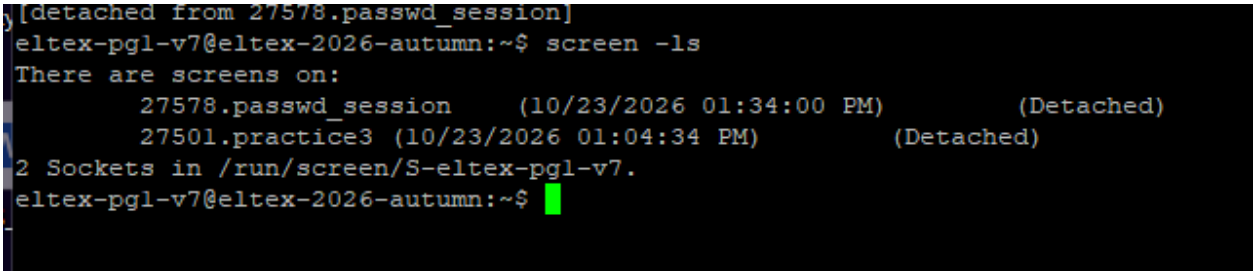



Рисунок 11 – Отсоединение от сессии passwd_session на сервере практики 1

***Комментарий:*** *Обе сессии (passwd_session и practice3) находятся в состоянии Detached.*

**Действие в top:**

```
фильтрация процессов по пользователю user1
```

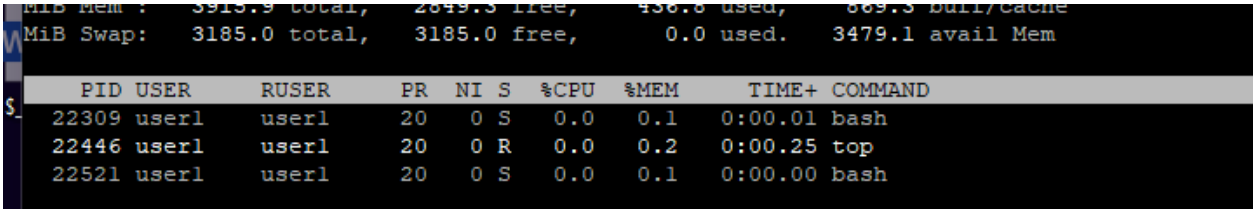



Рисунок 12 – Процессы пользователя user1 в top

***Комментарий:*** *В списке только процессы user1 (bash, top). Процесса passwd здесь нет: он принадлежит root, так как passwd работает с SUID-битом.*

**Действие в top:**

```
поиск процесса passwd
```

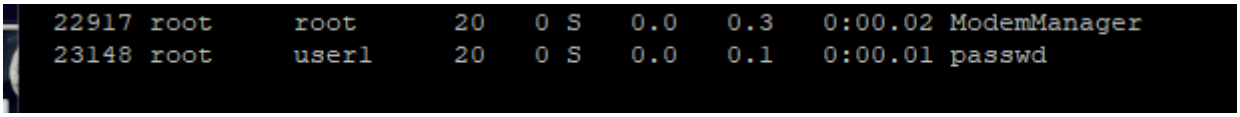



Рисунок 13 – Процесс passwd в списке top

***Комментарий:*** *Для passwd USER равен root (эффективный пользователь), RUSER равен user1 (реальный пользователь). PID процесса — 23148.*

**Клавиша top:**

```
k
```

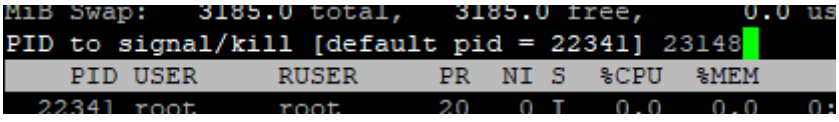



Рисунок 14 – Отправка сигнала из top: ввод PID процесса passwd

***Комментарий:*** *Клавиша k запрашивает PID процесса, затем номер сигнала.*

**Клавиша top:**

```
k
```

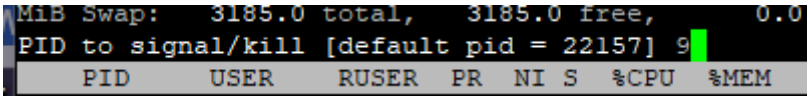



Рисунок 15 – Отправка сигнала из top: ввод числового идентификатора сигнала

***Комментарий:*** *Сигнал задаётся числом: 15 — SIGTERM, 2 — SIGINT, 3 — SIGQUIT, 9 — SIGKILL (см. man 7 signal).*

## 1.4. Задания (jobs), приоритет nice и сигналы

**Команды:**

```
vim ~/file_task3.txt
Ctrl+Z
sleep 600
Ctrl+Z
jobs
```

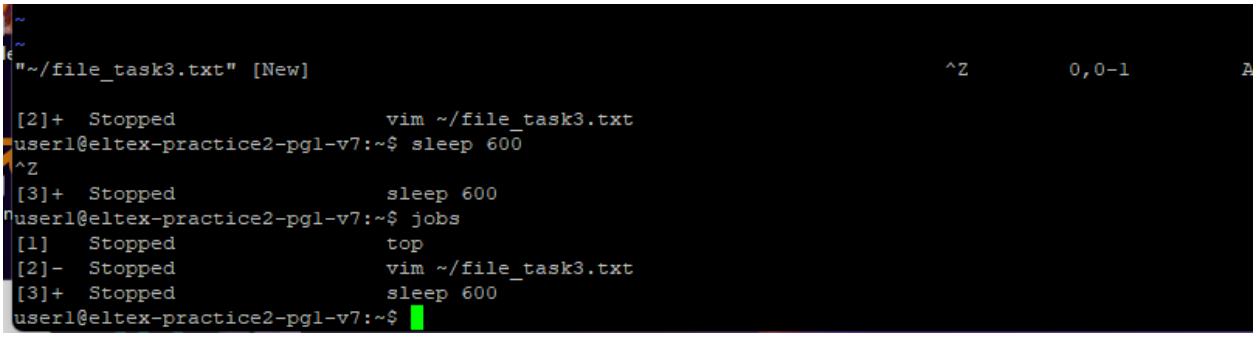



Рисунок 16 – Приостановка vim и sleep 600 и список заданий

***Комментарий:*** *Ctrl+Z приостанавливает задание (состояние Stopped). В списке три задания: [1] top, [2] vim ~/file_task3.txt, [3] sleep 600.*

**Команды:**

```
bg %3
pgrep sleep
renice 10 -p 23175
ps -o pid,ni,cmd -C sleep
```

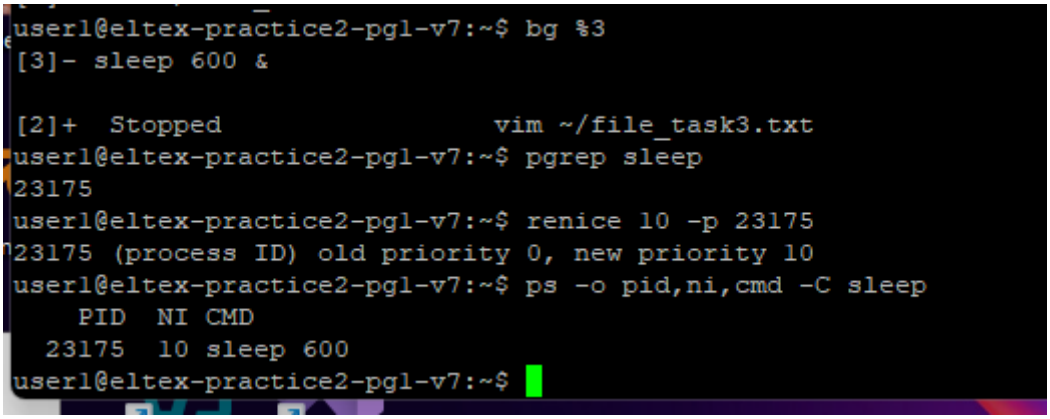



Рисунок 17 – Перевод sleep 600 в фон и изменение значения nice

***Комментарий:*** *bg %3 продолжает задание в фоновом режиме. renice изменил приоритет с 0 на 10, ps подтверждает: в столбце NI значение 10.*

**Команды:**

```
fg %2
ESC, :q!
kill -15 %3
jobs
```

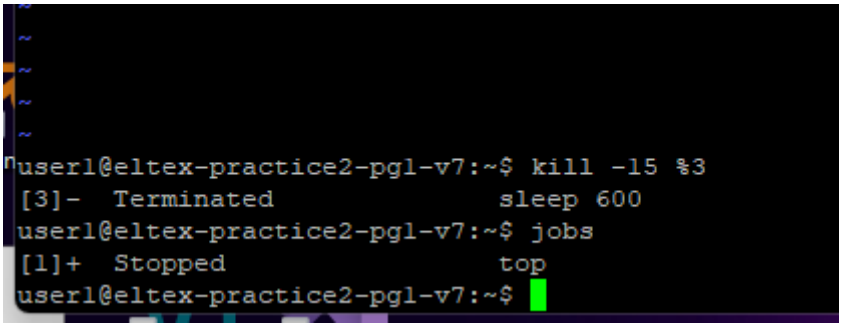



Рисунок 18 – Возврат к vim, выход из редактора и отправка SIGTERM заданию sleep 600

***Комментарий:*** *Задание sleep 600 завершено («Terminated»). В списке jobs остался только top.*

## 1.5. Перехват сигналов командой trap

**Команды:**

```
trap 'echo "Меня голыми руками не возьмёшь!"' SIGINT SIGQUIT
kill -2 $$
kill -3 $$
```

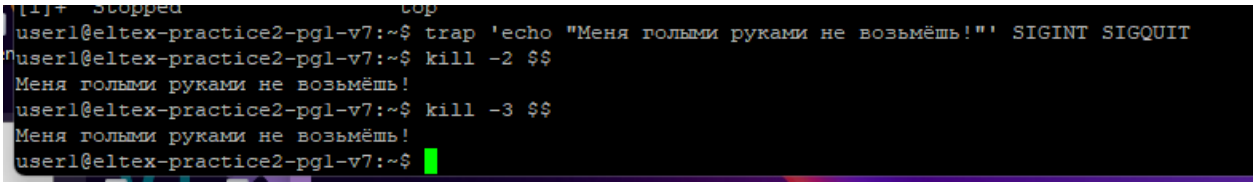



Рисунок 19 – Перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора

***Комментарий:*** *Переменная $$ содержит PID текущей оболочки. На каждый из сигналов (2 — SIGINT, 3 — SIGQUIT) оболочка вывела сообщение и продолжила работу.*

## 1.6. Зомби-процессы

**Команды:**

```
(sleep 1 & exec /bin/sleep 30) &
ps -eo pid,ppid,stat,cmd | grep 'Z'
```

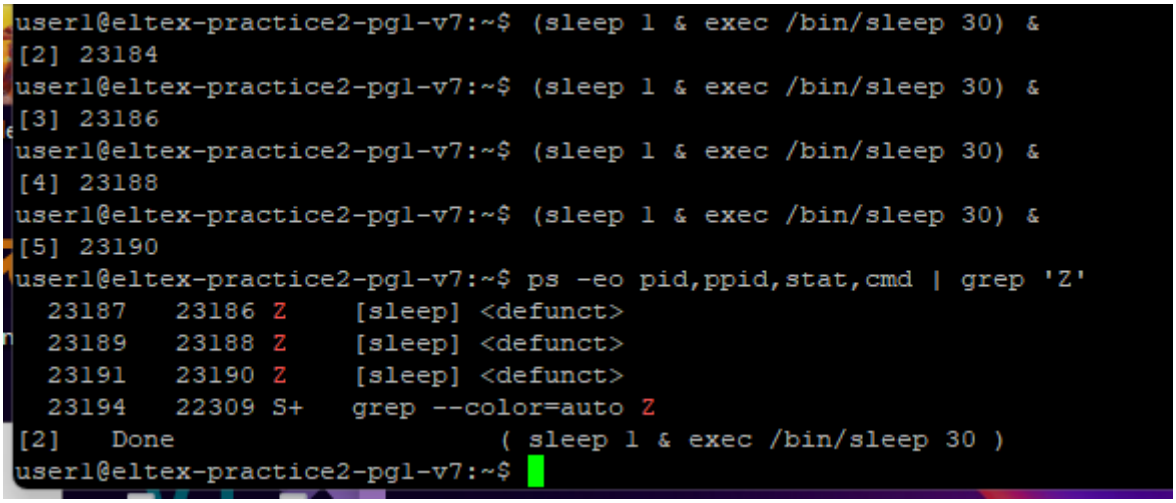



Рисунок 20 – Создание зомби-процессов и их поиск командой ps

***Комментарий:*** *Команда запущена несколько раз. В выводе ps процессы в состоянии Z ([sleep] <defunct>) с PID и PPID родителя. Родитель (sleep 30) не опрашивает завершившегося потомка, поэтому тот остаётся зомби.*

**Команды:**

```
ps -eo pid,ppid,stat,cmd | grep 'Z'
kill -9 23187
ps -eo pid,ppid,stat,cmd | grep 'Z'
```

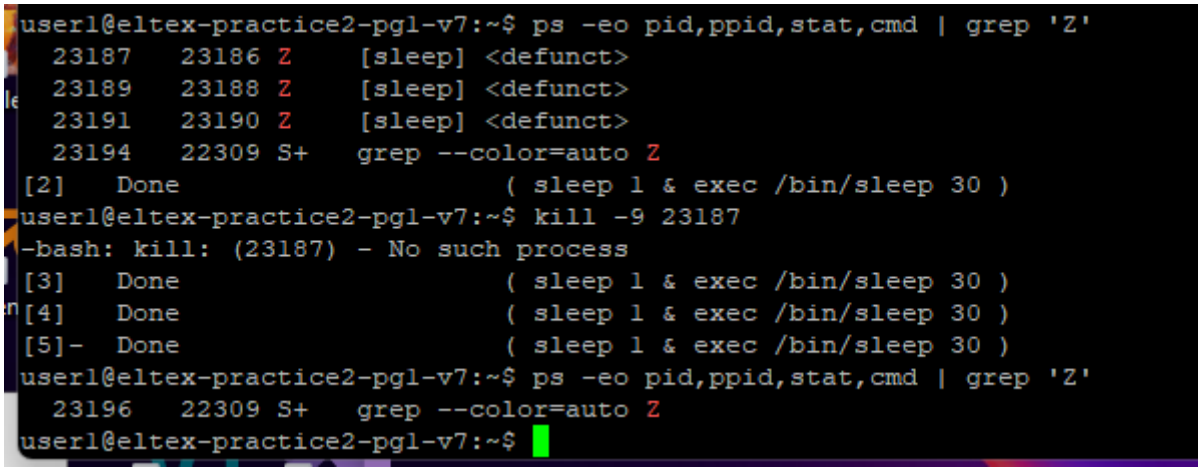



Рисунок 21 – Попытка досрочно завершить зомби-процесс сигналом SIGKILL

***Комментарий:*** *kill вернул «No such process»: к этому моменту родительские процессы sleep 30 завершились (jobs показывает Done), и зомби были убраны из таблицы процессов. Повторный поиск зомби ничего не находит.*

Зомби-процесс (состояние Z) — это процесс, который уже завершил свое выполнение, но все еще занимает запись в таблице процессов ядра. Это происходит потому, что родительский процесс еще не вызвал системный вызов wait() для получения кода завершения дочернего процесса. Зомби не потребляют оперативную память или процессорное время, так как их адресное пространство уже освобождено, но они занимают уникальный идентификатор PID. В ходе выполнения практики выяснилось, что зомби-процессы невозможно принудительно завершить отправкой сигналов, даже с помощью SIGKILL (сигнал 9), так как процесс уже мертв. Единственный способ избавиться от зомби — завершить его родительский процесс (PPID). После смерти родителя зомби-процессы становятся приемными детьми процесса init или systemd (PID 1), который автоматически собирает их коды возврата и очищает таблицу процессов. Если зомби-процессов накапливается слишком много, они могут исчерпать лимит доступных PID, что приведет к невозможности запуска новых процессов. Поэтому важно, чтобы программы, создающие дочерние процессы (через fork), корректно обрабатывали их завершение.

# Раздел 2. Скрипты bash, systemd и cron

## 2.1. Скрипт cuckoo.sh

**Команды:**

```
mkdir -p /root/practice3
cd /root/practice3
```

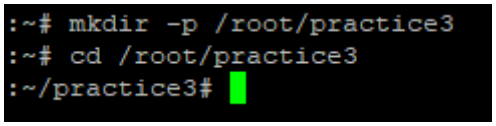



Рисунок 22 – Создание рабочего каталога

**Команда:**

```
nano cuckoo.sh
```

```
 GNU nano 7.2                                  cuckoo.sh *
#!/bin/bash

# 1. Парсим аргумент --pishitesuda=
for arg in "$@"; do
    case $arg in
        --pishitesuda=*)
        PIPE_PATH="${arg#*=}"
        ;;
    esac
done

if [ -z "$PIPE_PATH" ]; then
    echo "Ошибка: не указан путь к каналу (--pishitesuda=)"
    exit 1
fi

LOG_FILE="cuckoo.log"

# 2. Создаем именованный канал (FIFO)
mkfifo "$PIPE_PATH" 2>/dev/null

# 3. Записываем Startup
echo "$(date '+%Y-%m-%d %H:%M:%S') Startup!" >> "$LOG_FILE"

# 4. Перехватываем сигнал SIGTERM для корректного завершения
trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> "$LOG_FILE"; rm -f "$PIPE_PATH"; exit 0' SIGTERM
```

Рисунок 23 – Скрипт cuckoo.sh: разбор параметра, создание канала, запись Startup!

***Комментарий:*** *Скрипт получает путь к каналу из параметра --pishitesuda=, создаёт именованный канал (mkfifo), записывает в cuckoo.log сообщение Startup! и устанавливает обработчик сигнала SIGTERM, который пишет Shutdown!, удаляет канал и завершает работу.*

**Команда:**

```
nano cuckoo.sh
```

```
trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> "$LOG_FILE"; rm -f "$PIPE_PATH"; exit 0' SIGTERM

# 5. Бесконечный цикл чтения из канала
while true; do
    if read -r line < "$PIPE_PATH"; then
        # Проверяем, что формат сообщения верный: NAME[PID]: how much time do I have?
        if [[ "$line" =~ ^(.*)\[([0-9]+)\]:\ how\ much\ time\ do\ I\ have\?$ ]]; then
            NAME="${BASH_REMATCH[1]}"
            PID="${BASH_REMATCH[2]}"

            # Генерируем N от 2 до 10
            N=$((RANDOM % 9 + 2))

            # Логируем
            echo "$(date '+%Y-%m-%d %H:%M:%S') $NAME[$PID] $N" >> "$LOG_FILE"

            # Отправляем ответ обратно в канал
            echo "$N" > "$PIPE_PATH"
        fi
    fi
done
```

Рисунок 24 – Скрипт cuckoo.sh: цикл чтения запросов из канала

***Комментарий:*** *В бесконечном цикле скрипт читает строки из канала и проверяет формат «NAME[PID]: how much time do I have?». Число N от 2 до 10 вычисляется как RANDOM % 9 + 2, запрос и ответ записываются в cuckoo.log, ответ отправляется обратно в канал.*

**Команда:**

```
chmod +x cuckoo.sh
```

```
e3# chmod +x cuckoo.sh
e3#
```

Рисунок 25 – Права на выполнение для cuckoo.sh

## 2.2. Служба systemd для cuckoo.sh

**Команда:**

```
nano /etc/systemd/system/cuckoo.service
```

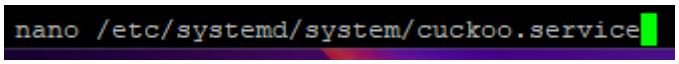



Рисунок 26 – Создание файла службы

**Команда:**

```
nano /etc/systemd/system/cuckoo.service
```

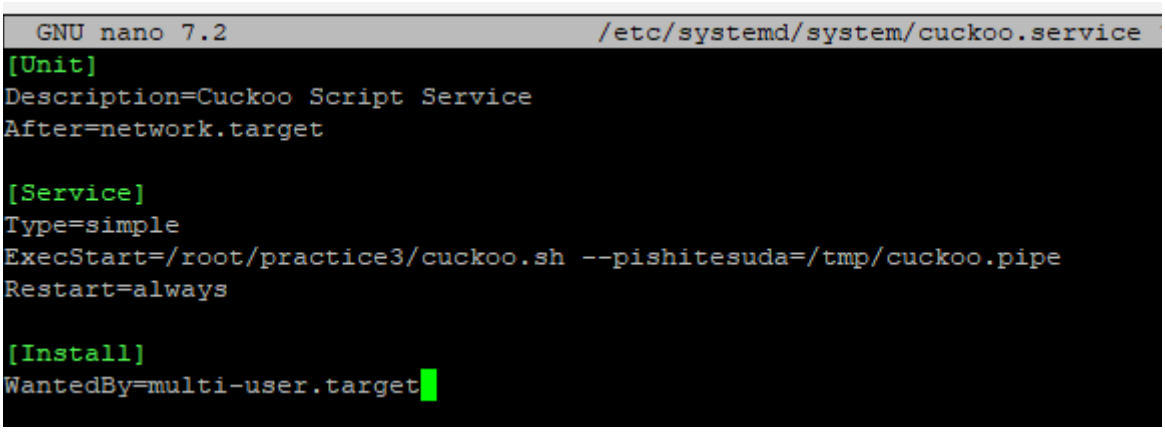



Рисунок 27 – Содержимое файла cuckoo.service

***Комментарий:*** *ExecStart запускает скрипт с параметром --pishitesuda=/tmp/cuckoo.pipe, Restart=always перезапускает службу при остановке, WantedBy=multi-user.target включает автозапуск.*

**Команды:**

```
systemctl daemon-reload
systemctl enable cuckoo.service
systemctl start cuckoo.service
systemctl status cuckoo.service
```

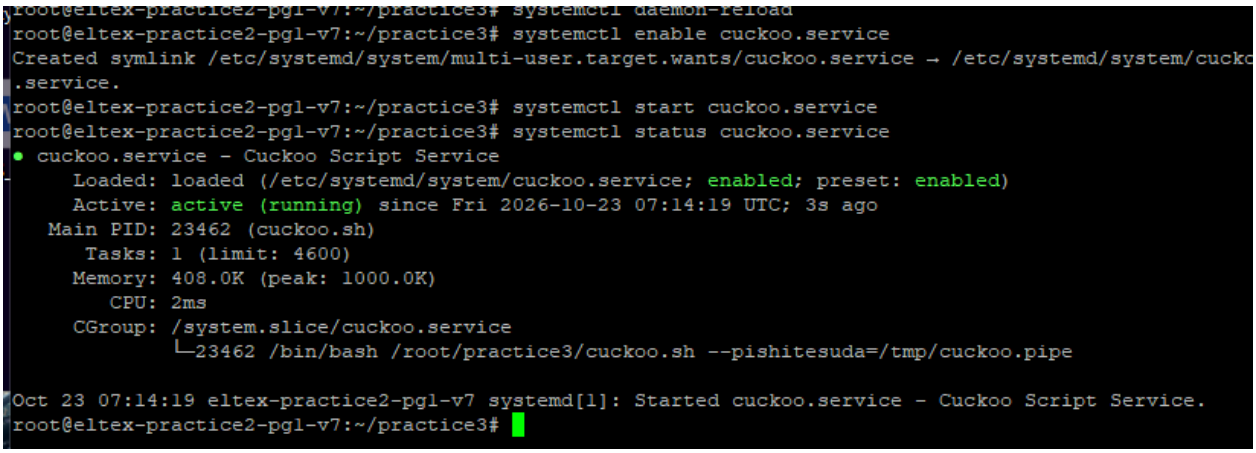



Рисунок 28 – Включение, запуск и проверка состояния службы cuckoo

***Комментарий:*** *Служба загружена (enabled) и активна (active (running)); в cgroup работает процесс /bin/bash /root/practice3/cuckoo.sh.*

## 2.3. Скрипт template_task.sh

**Команда:**

```
nano template_task.sh
```

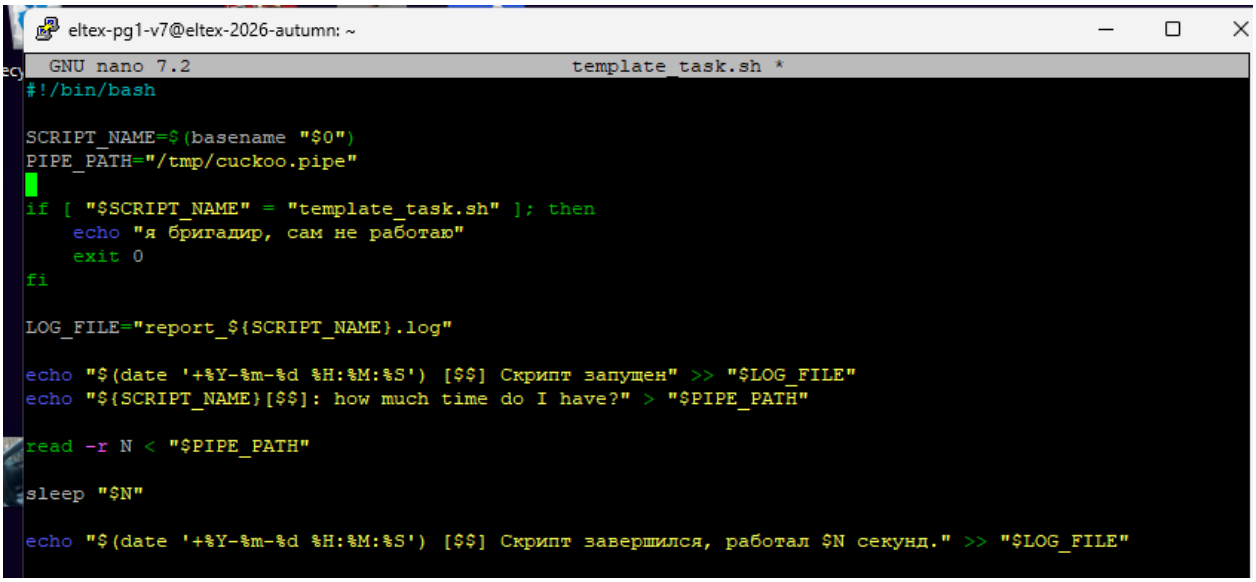



Рисунок 29 – Скрипт template_task.sh

***Комментарий:*** *Скрипт проверяет своё имя: при запуске под именем template_task.sh выводит «я бригадир, сам не работаю» и выходит. Иначе он записывает в report_имя.log время запуска, отправляет запрос в канал cuckoo, читает число N, ждёт N секунд и записывает сообщение о завершении.*

**Команда:**

```
ln -s /root/practice3/template_task.sh /root/practice3/worker_test
```

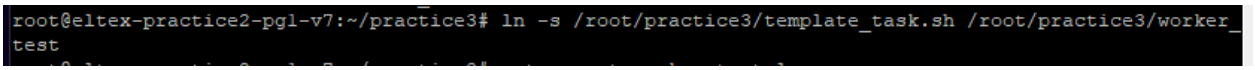



Рисунок 30 – Символическая ссылка worker_test на template_task.sh

***Комментарий:*** *Ссылка позволяет запускать один и тот же скрипт под разными именами (рабочие задачи).*

**Команды:**

```
/root/practice3/worker_test
cat report_worker_test.log
```

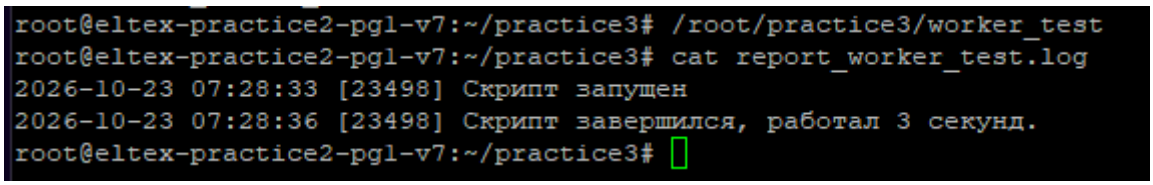



Рисунок 31 – Запуск задачи worker_test и её отчёт

***Комментарий:*** *Отчёт содержит сообщения о запуске и завершении: скрипт получил от cuckoo число 3 и проработал 3 секунды.*

## 2.4. Скрипт observer.sh

**Команда:**

```
nano observer.sh
```

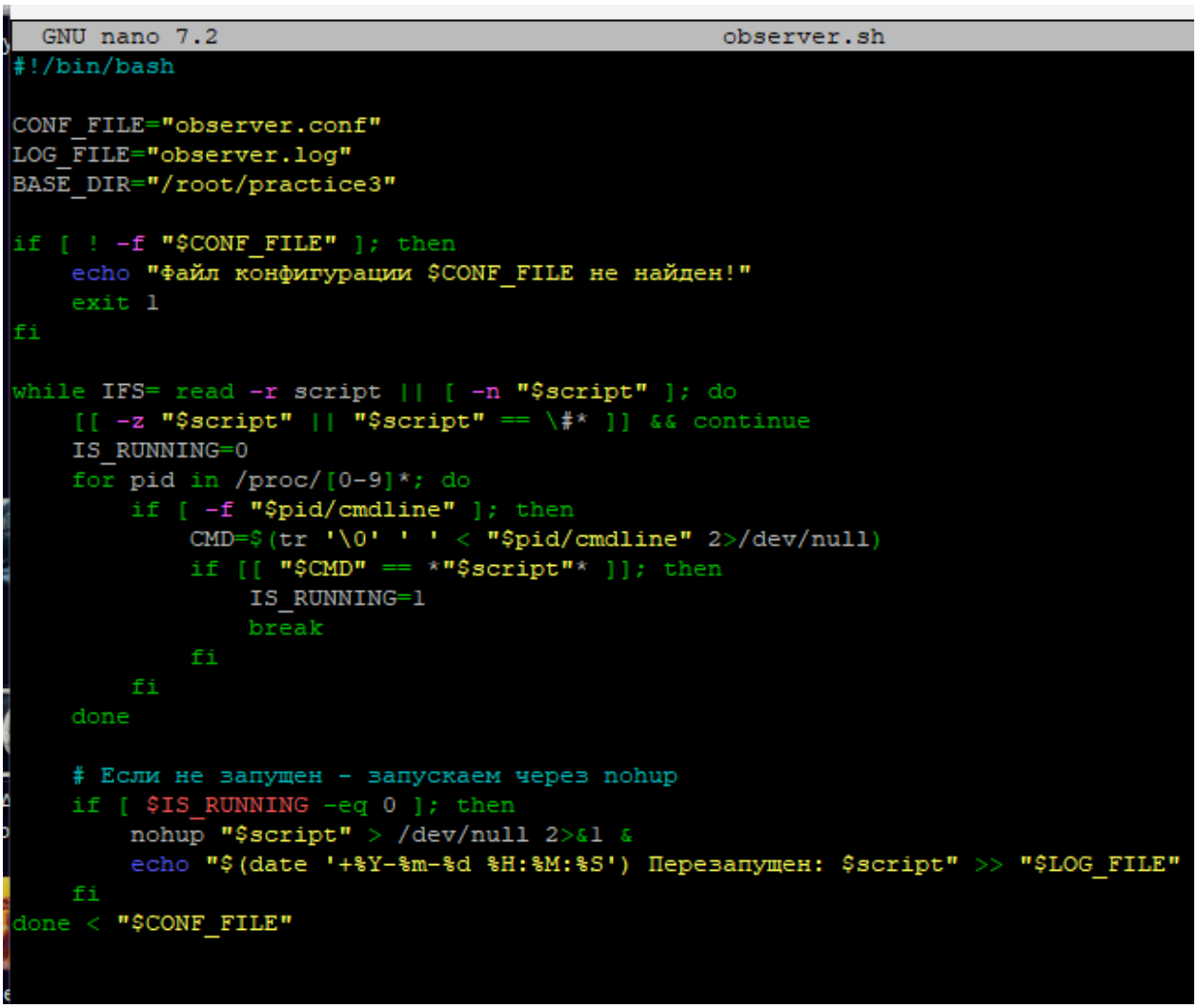



Рисунок 32 – Скрипт observer.sh

***Комментарий:*** *Скрипт читает список скриптов из observer.conf, проверяет по /proc/*/cmdline, запущен ли каждый из них, и в случае отсутствия запускает его через nohup в отключённом от терминала режиме. Информация о перезапуске дописывается в observer.log.*

## 2.5. Запуск observer.sh по расписанию cron

**Команда:**

```
crontab -e
```

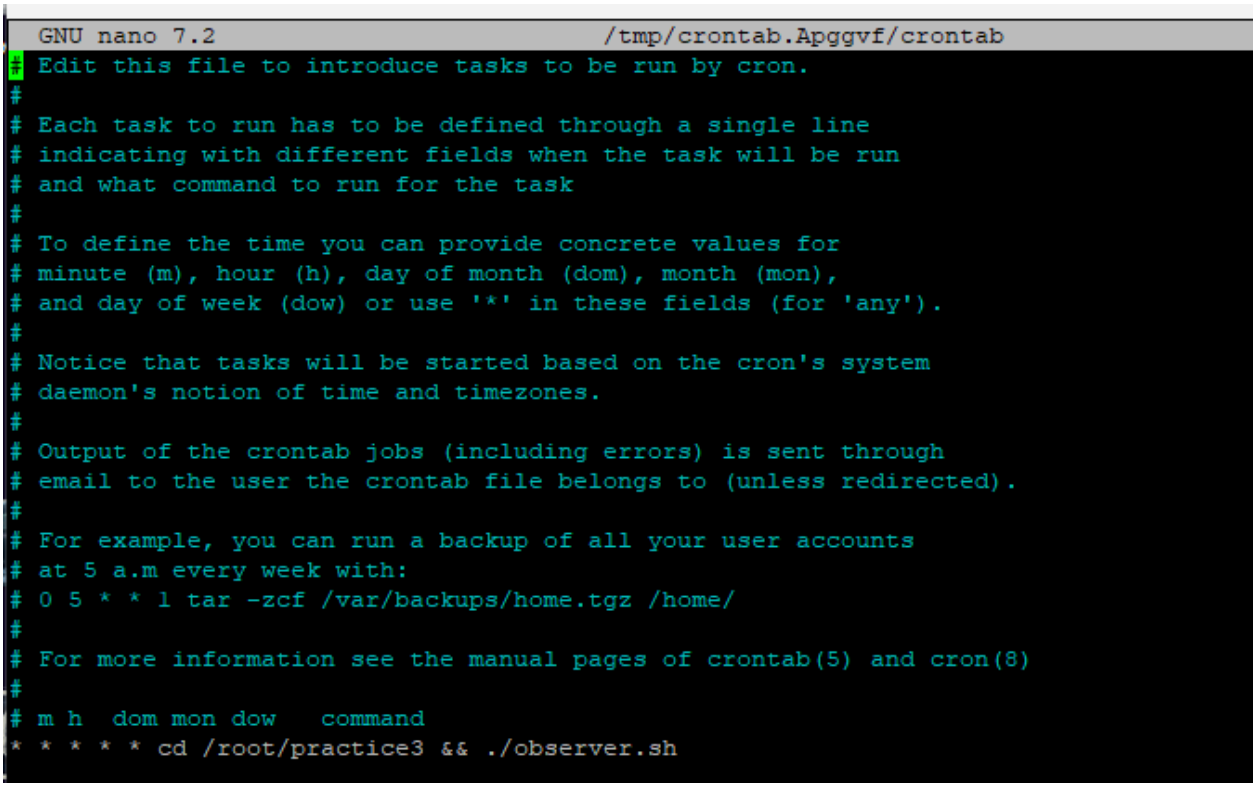



Рисунок 33 – Добавление задачи в crontab: запуск observer.sh раз в минуту

***Комментарий:*** *Строка «* * * * * cd /root/practice3 && ./observer.sh» запускает скрипт каждую минуту. Перед запуском выполняется переход в рабочий каталог.*

## 2.6. Рабочие задачи и конфигурация observer

**Команды:**

```
ln -s /root/practice3/template_task.sh /root/practice3/task_alpha
ln -s /root/practice3/template_task.sh /root/practice3/task_beta
```

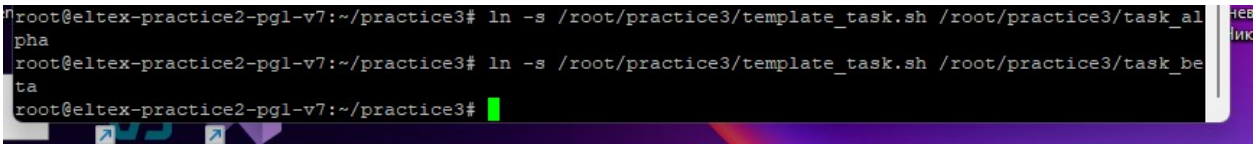



Рисунок 34 – Создание символических ссылок task_alpha и task_beta на template_task.sh

**Команда:**

```
nano observer.conf
```

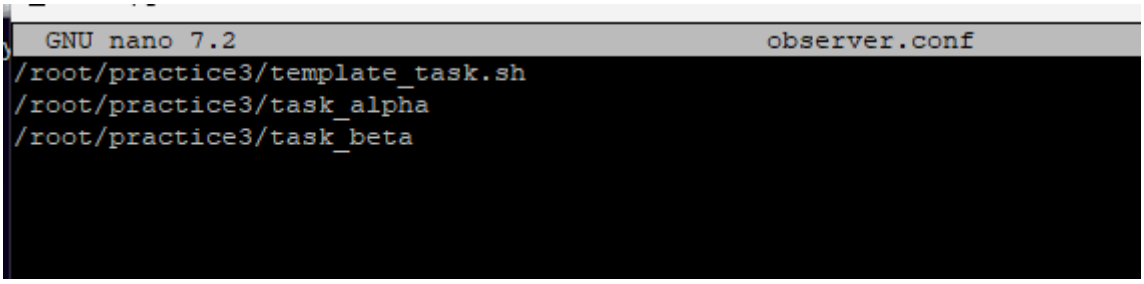



Рисунок 35 – Файл конфигурации observer.conf

***Комментарий:*** *В файле перечислены пути к исходному скрипту template_task.sh и к ссылкам task_alpha и task_beta.*

**Команда:**

```
ls -ls
```

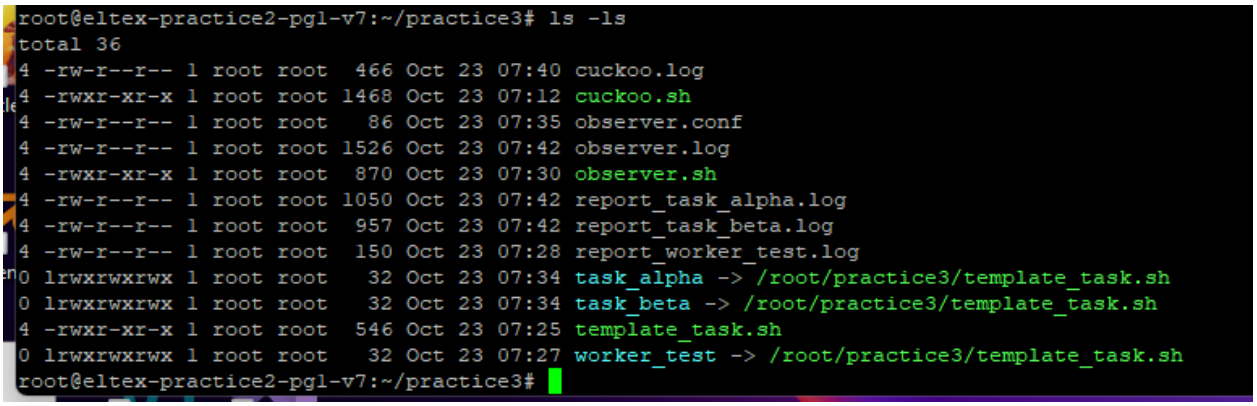



Рисунок 36 – Итоговый набор файлов в рабочем каталоге

***Комментарий:*** *Скрипты cuckoo.sh, template_task.sh и observer.sh имеют права на выполнение; есть конфигурация observer.conf, журналы cuckoo.log, observer.log и report_*.log; task_alpha, task_beta и worker_test являются символическими ссылками (тип l) на template_task.sh.*

**Команда:**

```
cat observer.log
```

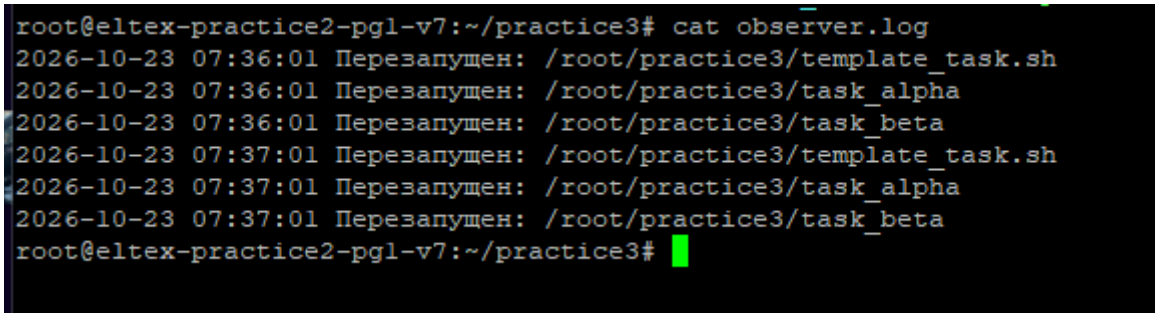



Рисунок 37 – Журнал перезапусков observer.log

***Комментарий:*** *Записи о перезапуске всех трёх скриптов из observer.conf появляются раз в минуту, как задано расписанием cron.*

## 2.7. Архив с результатами

**Команда:**

```
tar -czvf practice3_task2.tar.gz *.sh *.conf report_*.log observer.log
cuckoo.log
```

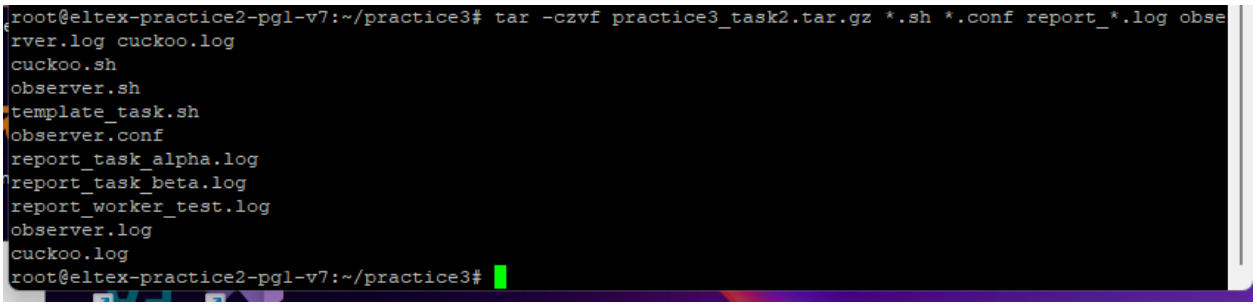



Рисунок 38 – Создание сжатого архива с логами и исходными текстами скриптов

***Комментарий:*** *Ключи -czvf: создать архив, сжать gzip, показать список файлов, указать имя архива.*

**Команда:**

```
ls -ls
```

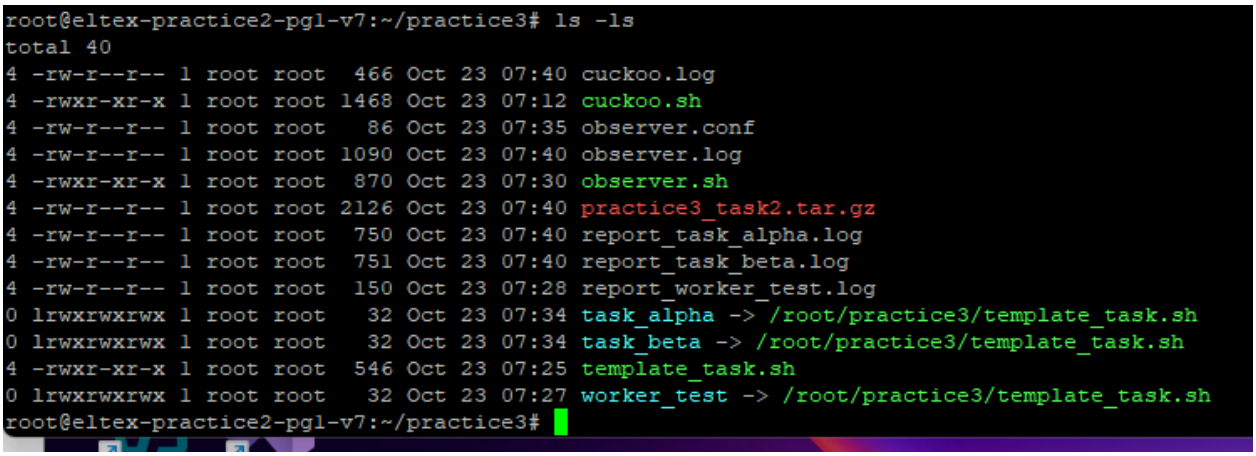



Рисунок 39 – Архив practice3_task2.tar.gz в рабочем каталоге

## 2.8. Остановка процессов

**Команды:**

```
systemctl stop cuckoo.service
systemctl disable cuckoo.service
```

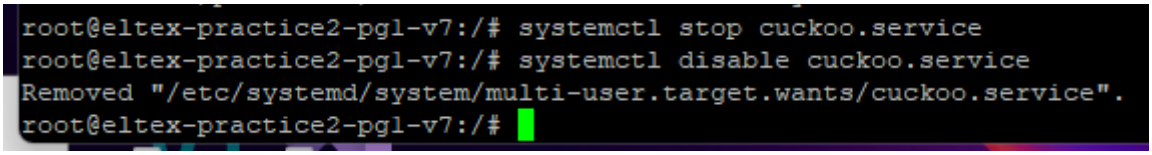



Рисунок 40 – Остановка и запрет службы cuckoo

***Комментарий:*** *Команда disable убрала ссылку автозапуска из multi-user.target.wants.*

**Команда:**

```
crontab -e
```

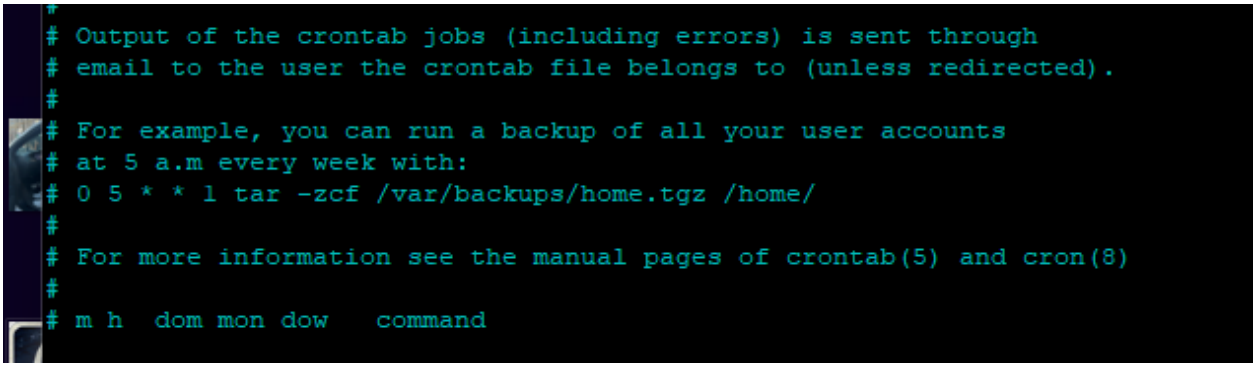



Рисунок 41 – Удаление задачи observer.sh из crontab

***Комментарий:*** *В файле остались только комментарии, строка с запуском observer.sh удалена.*

**Команды:**

```
pkill -f template_task.sh
pkill -f task_alpha
pkill -f task_beta
```

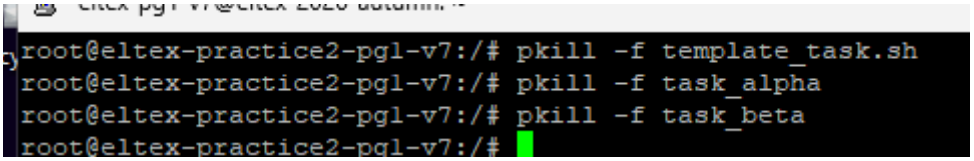



Рисунок 42 – Завершение оставшихся фоновых процессов

***Комментарий:*** *Ключ -f ищет процессы по полной командной строке, поэтому находятся и запущенные через символические ссылки.*